

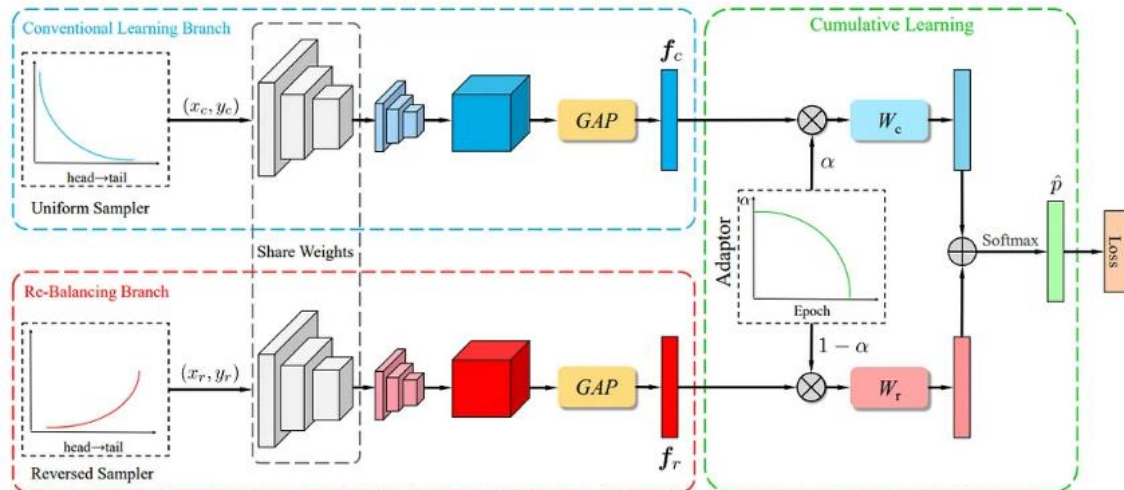
# CVPDL hw2 Object Detection Report

School：國立清華大學 National Tsing Hua University

Student ID：114064545

Name：謝尚哲

## 1. Model Description



▲ Fig1. BBN: Bilateral-Branch Network With Cumulative Learning for Long-Tailed Visual Recognition (CVPR2020)

與 hw1 不同，這次除了要辨識四個類別 {car, motorcycle, person, hov}，最有挑戰的是資料呈現 **long-tailed** 分布，為了解決這個問題，我參考了於 CVPR 2020 發表的 BBN (Bilateral-Branch Network) 架構 (見 Fig.1)。

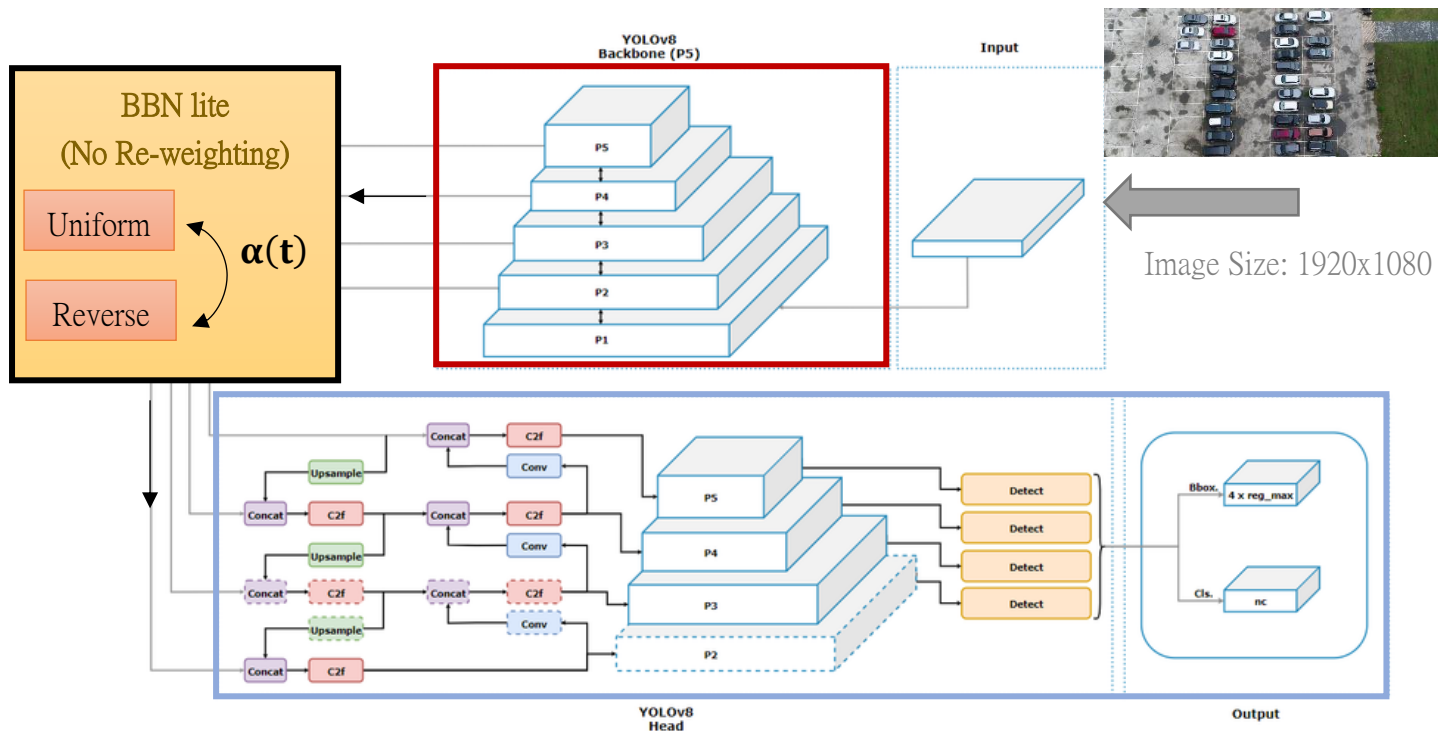
在原論文中，BBN 將輸入資料分為兩個分支：

- **Conventional Learning Branch (Uniform Sampler)：**用原始資料分布訓練以學習整體特徵。
- **Re-Balancing Branch (Reversed Sampler)：**重新取樣資料，增加 tail 類別的比例，讓模型更重視稀有的樣本。

為了使 BBN 能夠讓 training data 套入並且成功銜接 YOLOv8 架構，我設計了簡化版的 **BBN-Lite** (見 Fig.2)。不同於原始 BBN 將網路拆成雙分支，我只用了單一路徑 (YOLOv8 backbone + head)：

- 使用 DataLoader 的 sampling 機制，分成 Uniform Sampler 和 Reversed Sampler
- 在每個 epoch 開始時根據  $\alpha$  調整取樣比例，使模型於訓練早期偏向 head 類別、後期則逐漸聚焦於 tail 類別

- 因為只有單一路徑 YOLOv8，兩者是一起訓練，因此也巧妙地實現 BBN 架構(Fig1.)中 weight sharing 的部分（implicit weight sharing）。
- 避免模型過於複雜且確保訓練時穩定並且 GPU memory 不會過大，因此刪除 Re-Weighting



▲Fig2. YOLO v8 + BBN lite architecture

整體而言，我所設計的 YOLOv8+BBN lite 架構 可分為三個主要部分：

### • Backbone

- 負責從輸入影像中提取多層次的特徵資訊。
- 基於 CSPDarknet 架構，並引入 C2f 模組（Cross-Stage Partial connections with Faster implementation），該模組透過卷積後將特徵圖切分成兩部分，再進行部分特徵融合，以兼顧運算效率與表現。
- 採用了 SPPF（Spatial Pyramid Pooling – Fast）取代傳統 FPN，能在不同感受野下聚合多尺度資訊，提升對大小物體的辨識能力。
- 輸出三個不同解析度的特徵圖（P3, P4, P5），分別對應於不同層級的目標偵測需求。

### • BBN-lite Module

| BBN Module                                          | BBN-lite Module                                             |
|-----------------------------------------------------|-------------------------------------------------------------|
| 兩個分支，一個 head 資料多，另一個 tail 資料多，最後將兩者訓練結果以 alpha 調整比例 | 單一路徑，訓練前期主要看 head，後期主要看 tail，這裡的 alpha 控制看 head 和看 tail 的比例 |

- **Head**

- **Bounding Box Regression (bbox)**：預測物體在影像中的位置與尺寸
- **Object Classification (cls)**：判斷物體所屬的類別。
- 採用 **Decoupled Head** 結構，將分類與定位任務分離以提升訓練穩定度。Head 會在三個尺度上（對應 backbone 的 P3、P4、P5）同時進行偵測，以適應不同大小的物體，輸出最終的 **cls, bbox, conf** 偵測結果。

## 2. Implementation Details

- **Dataset**



- **Data Preprocessing**

- 採 YOLO letterbox 的方式在訓練時縮放到 **imgsz=1280**（以灰邊做補齊），改善過程之中也在預測時縮放到 **imgsz=1536**（即 training 時 1280、predicting 時 1536）
- 訓練階段未使用任何資料增強 — 關閉幾何與色彩增強，在不影響 **BBN-Lite re-balancing** 的過程完成訓練，避免對小目標的擾動。
- 使用 YOLO loss functions – bbox loss, cls loss, DFL loss

- **Training Strategies**

- 使用 **YOLOv8x** 為基礎架構：backbone 較 YOLOv8n/s/m 更深、更寬，具備更強的特徵擷取能力
- 結合 **BBN-Lite**：為了解決 long-tailed 資料分布而使用，別於原始的 BBN Module，BBN-lite 根據 alpha 調整取樣比例，訓練初期以 head 類別為主，後期逐漸提升 tail 類別比例，使模型能在單一訓練流程中達到 head & tail 平衡學習。
- **AdamW 優化器**：提供自適應學習率與解耦權重衰減機制，能在長時間訓練下保持穩定與平滑的收斂。
- **混合精度訓練 (AMP)**：可同時降低 GPU 記憶體使用量並加快訓練速度
- **餘弦學習率衰減 (Cosine Learning Rate Decay)**：讓學習率在訓練過程

中平滑下降，避免突變造成的訓練震盪。

### 3. Result Analysis

- Quantitative Improvements / Hyperparameter Settings

**\*Reverse Gamma / Reverse Target:** 簡化 BBN 的 Reverse Sampling Weights，設計  $w_c$  用來定義類別權重，其中

$$c \in \{\text{car, hov, person, motorcycle}\}$$

$$w_c = \left( \frac{n_{target}}{n_c} \right) - \gamma$$

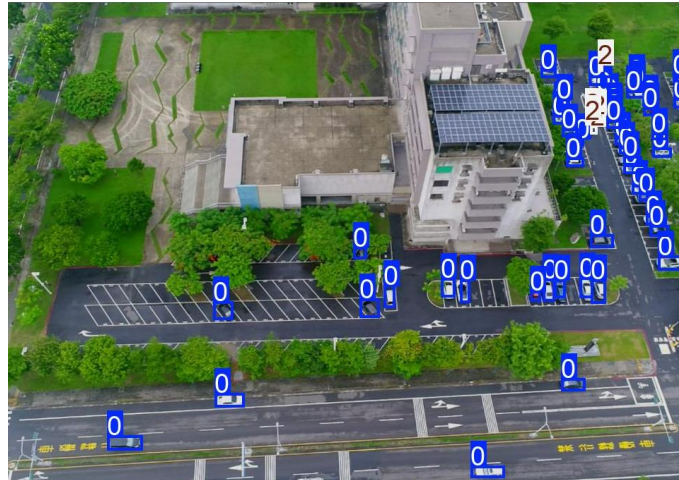
# of submission: 35 times

| Model                                                  | Settings (w/o pretrained weight & feature extractor)                                                                                                                          | F1-Score       |
|--------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------|
| YOLOv8n                                                | Epoch=120, imgsz=640, optimizer=AdamW conf=0.01                                                                                                                               | 0.13663        |
| YOLOv8n + <b>BBN-lite</b>                              | <b>Epoch=240, imgsz=1280, lr0=0.002, lrf=0.01</b><br><b>weight_decay=0.01</b> optimizer=AdamW <b>conf=0.005</b> /<br><b>*REVERSE_GAMMA=0.7,</b><br><b>REVERSE_TARGET=3000</b> | 0.24313        |
| YOLOv8n + BBN-lite                                     | Epoch=240, imgsz=1280, lr0=0.002, lrf=0.01<br>weight_decay=0.01 optimizer=AdamW conf=0.005<br><b>augment=True</b> / REVERSE_GAMMA=0.7,<br>REVERSE_TARGET=3000                 | 0.24152        |
| <b>YOLOv8x</b> + BBN-lite                              | Epochs=240, imgsz=1280, lr0=0.002,lrf=0.01 conf=0.005<br>/ REVERSE_GAMMA=0.7, REVERSE_TARGET=3000                                                                             | 0.25380        |
| YOLOv8x + BBN-lite                                     | Epochs=240, <b>training_imgsz=1280,</b><br><b>predict_imgsz=1536</b> lr0=0.002,lrf=0.01 conf=0.005<br>/ REVERSE_GAMMA=0.7, REVERSE_TARGET=3000                                | 0.25991        |
| YOLOv8x + BBN-lite                                     | Epochs=240, training_imgsz=1280, predict_imgsz=1536<br>lr0=0.002,lrf=0.01 <b>conf=0.0005</b><br>/ REVERSE_GAMMA=0.7, REVERSE_TARGET=3000                                      | 0.26030        |
| YOLOv8x + BBN-lite                                     | Epochs=240, training_imgsz=1280, predict_imgsz=1536<br>lr0=0.002,lrf=0.01 conf=0.0005 <b>max_det=300</b><br>/ REVERSE_GAMMA=0.7, REVERSE_TARGET=3000                          | 0.26030        |
| 發現 YOLOv8x 預設的 max_det 就是 300，因此開始調整不同值對 prediction 影響 |                                                                                                                                                                               |                |
| YOLOv8x + BBN-lite                                     | max_det=400                                                                                                                                                                   | 0.26213        |
|                                                        | max_det=700                                                                                                                                                                   | 0.26448        |
|                                                        | max_det=900                                                                                                                                                                   | 0.26497        |
|                                                        | max_det=1200                                                                                                                                                                  | 0.26536        |
|                                                        | max_det=1200, tta=True(predict 階段)                                                                                                                                            | <b>0.27185</b> |

- Visualizations

### 1. Ground truth (e.g. img0631)

種類{0, 1, 2, 3} = {car, hov, person, motorcycle}



## 2. Predicted Image (e.g. img0003)



- **Addresses the long-tailed problem**

計算先前定義好的  $w_c$ ，若值越大，代表越容易被模型取樣，以下是計算結果：

| 類別            | 樣本數         | weight      |
|---------------|-------------|-------------|
| car           | 18315       | 0.28        |
| <b>hov</b>    | <b>1122</b> | <b>1.84</b> |
| <b>person</b> | <b>2900</b> | <b>1.02</b> |
| motorcycle    | 4604        | 0.69        |

←有了權重調整，使得訓練時 tail 類別 (hov, person) 更容易被 YOLO 模型取樣

|          |                |                |
|----------|----------------|----------------|
|          | Before         | After          |
| F1 score | <b>0.13663</b> | <b>0.24313</b> |

## 4. Short Conclusion

- **YOLOv8 架構+ BBN-Lite Re-Balancing Module**，成功解決 long-tailed 資料集中樣本不平衡的問題
- 採用**動態取樣**：1.  $\alpha(t)$ 隨著時間軸(epoch)變化 2.  $w_c$  (Reverse Sampling weight) 隨著類別變化
- 使用更複雜的 backbone(YOLOv8n -> YOLOv8x) 可以得到更好的表現
- 模型預設 max\_det 為 300，因此限制了 prediction 的發揮，將 max\_det 往上調整也獲取更好的成績



## 5. Reference

- [1] Zhou et al., *BBN: Bilateral-Branch Network with Cumulative Learning for Long-Tailed Visual Recognition* CVPR 2020.
- [2] BBN 論文解讀  
<https://medium.com/@XingChen/bbn-bilateral-branch-network-with-cumulative-learning-for-long-tailed-visual-recognition-%E8%AB%96%E6%96%87%E8%A9%B3%E7%B4%B0%E8%A7%A3%E8%AE%80-2491805342e4>
- [3] BBN GitHub 程式碼  
<https://github.com/megvii-research/BBN>
- [4] Detailed Explanation of YOLOv8 Architecture  
<https://medium.com/@juanpedro.bc22/detailed-explanation-of-yolov8-architecture-part-1-6da9296b954e>